

Network Resiliency and Concurrency Protection Profile

This document specifies the multi-tier network protection profile, client-side transport safeguards, and error-handling taxonomies implemented to guarantee high availability and prevent pipeline process hangs.

Network Resilience Features

The query processing systems and interactive communication loops implement a robust, multi-tier network protection profile to ensure high availability and prevent process hangs:

- **Low-Level Socket Timeout Enforcement:** To prevent silent TCP socket hangs or indefinite thread blocking, the central API client is configured with a strict 120-second (2 minutes) HTTP read timeout globally across all callers. If a socket stalls or drops silently under heavy network load, the connection is forcibly closed, throwing a clean exception that can be recovered from safely.
- **Jittered Exponential Backoff:** Symmetrically handles transient API capacity overloads (specifically HTTP 503 Service Unavailable) by executing up to 2 randomized exponential retries (3 total attempts). Symmetrically, any other HTTP error (including HTTP 504 Gateway Timeout, 429 Rate Limits, or ReadTimeouts) is treated as an unrecoverable failure, instantly halting execution and preserving drafts to ensure absolute database stability and prevent split-brain issues. The retry loop utilizes a randomized wait window (10.0 to 60.0 seconds) to perfectly stagger parallel requests, absorb server-side capacity spikes, and guarantee absolute loop stability.
- **Localized Error Isolation:** During multi-query batch sessions, the processor executes its queries within localized, query-level safety handlers. Symmetrically, if any query fails completely due to an unrecoverable connection drop after all retries are exhausted, the loop intercepts the error locally, records it in the output report, and gracefully continues processing the remaining queries in the file with zero crashes.
- **Conversational Fallback:** During user-driven conversational REPL sessions, the turn-level loop intercepts network drops locally. Symmetrically, if a connection drop occurs while transmitting a message, the loop catches the exception, prints

a turn failure warning to the console, and gracefully continues the terminal loop. Symmetrically, the conversational thread history remains intact, allowing the human operator to resubmit their question when the network recovers.

The 3-Category API Resiliency Taxonomy

To enforce quota safety, budget protection, and state integrity, the system categorizes all generative API communication failures and HTTP network drops into exactly three distinct resiliency classes:

Category A: Transient, Retry-Safe Errors

Temporary network-level, transport-layer, or edge proxy capacity drops. Symmetrically, retrying them after a staggered exponential backoff delay is completely safe, highly effective, and will hit healthy backend replicas: * **503 SERVICE_UNAVAILABLE** (Upstream model replica overload). * **502 BAD_GATEWAY** (Temporary edge routing drop). * **RemoteProtocolError** & **Server Disconnection** (Transient TCP connection drop before receiving HTTP response headers). * **Connection Reset by Peer & Broken Pipe** (Abrupt transport-level socket terminations under concurrent fanned-out network payloads). * **ReadError** & **Bad File Descriptor** (Socket-sharing anomalies and local network read/write failures under parallel load, handled safely via thread-local client caching and retry).

Category B: Explicitly Handled Unrecoverable Errors

System-level bottlenecks, quota ceilings, or fatal model crashes. Symmetrically, the pipeline will fail-fast immediately on these errors—preserving all drafted coordinate states, rolling back affected page census checklist items to pending on disk, and bubbling an explicit connection exception. Symmetrically, unrecoverable synthesis exceptions (such as Gemini gateway timeouts, rate limits, or collection read failures) and low-level visual fallback PDF rendering errors are propagated immediately upwards under the Fail-Fast paradigm rather than swallowed, triggering standard unescaped red FAIL milestones: * **504 GATEWAY_TIMEOUT** (Large context-weight generation timeout). * **429 RESOURCE_EXHAUSTED** (API rate limits exceeded; fail-fast immediately to respect quota cooling windows). * **500 INTERNAL_ERROR** (Fatal server-side model crash). * **JSONDecodeError** (Probabilistic LLM syntactic or JSON formatting anomalies, handled safely inside verifier threads via localized checklist rollbacks and queue preservation).

Category C: Unrecoverable API Error Fallback

Client-side parameter bugs, authentication issues, resource mismatches, or any other unclassified exceptions. Symmetrically, they are handled by the standard unrecoverable API error fallback block, mapping directly to unrecoverable fail-fast exceptions: * **400 INVALID_ARGUMENT** (Bad Request). * **403 PERMISSION_DENIED** (Authentication / API Key error). * **404 NOT_FOUND** (Resource or Model mismatch).

Centralized Exception Classification

Centralized Exception Wrapper

To enforce robust error propagation and prevent circular module loading dependencies inside our parent orchestrators and command-line interfaces, the network layers declare a dedicated custom exception class:

- **Logical Responsibility:** Symmetrically, this custom exception class (which inherits from Python's standard `RuntimeError`) serves as the single centralized wrapper for all unrecoverable network, edge proxy, and API gateway failures (such as Categories B and C, local read/connect timeouts, and socket drops).
 - **Preflight Exit Mapping:** Symmetrically, when an unrecoverable connection failure is caught, the system raises this exception. This is intercepted at the command-line interface level and mapped directly to a preflight exit code, terminating the process cleanly and avoiding verbose tracebacks.
 - **Complete Thread Separation Compatibility:** Symmetrically, because this class is defined inside our utility layers, it is completely decoupled from any orchestrators or CLI commands. This enables worker threads to fail gracefully and return without raising thread-level exceptions that would prematurely abort our independent parallel thread execution pools.
-

Symmetrical Exception-Handling Boundaries

To prevent redundancy and maintain a single source of truth under the Don't Repeat Yourself (DRY) principle, the precise code-level exception-handling boundaries, thread events, and localized handlers are documented in [docs/error_handling.md](#).

To review how each specific pipeline step responds to Category A, B, and C exceptions under their respective execution classes, see the complete pipeline

resilience matrix in [docs/resilience_table.md](#).